REOAGMS: Presentation script

This script gives the **exact words** each member says. The team deck `REOAGMS-Team-Presentation.pptx` has **10 slides** and runs together with the live console as one **5-minute** demo on one laptop. Each member speaks while their slide is up and switches to the app for the action shown under *On screen*. The same words are in the slides' speaker notes.

Before you start

- Stack running (`docker compose up -d`), browser signed in as `admin@reoagms.com` on http://localhost:5173, a second tab on Grafana. Zoom 100 %, notifications silenced.
- Two minutes earlier, open the Alerts page and resolve the open alerts; the simulator then raises fresh events within about a minute.
- Rehearse with a stopwatch. Hand over in one sentence, as written.
- If the fault has not arrived by 1:30, Govindu says "the simulator injects one every minute" and Thurunu uses the newest order already on the board.

Credentials

Use	Login
Console (all roles)	`admin@reoagms.com` , `manager@reoagms.com` , `operator@reoagms.com` , `engineer@reoagms.com` , `executive@reoagms.com` · password `Reoagms@2026`
Grafana http://localhost:3001	user `admin` · password `reoagms`
RabbitMQ http://localhost:15672	`reoagms` / `reoagms_password`
Prometheus http://localhost:9090	no login

Speaking order and time

Slide	Time	Speaker
1	0:00–0:10	Charith
2	0:10–0:45	Charith
3	0:45–1:15	Nemal
4	1:15–2:00	Govindu
5	2:00–2:35	Thurunu
6	2:35–3:05	Yasiru
7	3:05–3:45	Sachinthana

Slide	Time	Speaker
8	3:45–4:15	Charith
9	4:15–4:50	All members
10	4:50–5:00	All members

Total spoken text: about 509 words, which is under 5 minutes at a calm pace, leaving a few seconds for clicks.

The exact words

Slide 1 · 0:00–0:10 · Charith

On screen: Title slide

Charith says: Good morning. We are presenting REOAGMS, a platform that monitors and maintains renewable energy assets in real time.

Slide 2 · 0:10–0:45 · Charith

On screen: Console: sign in as admin, then the Control room

Charith says: REOAGMS is six Spring Boot microservices behind one API gateway, connected by RabbitMQ events, each with its own PostgreSQL database. I sign in as the administrator; the gateway validates my token and role. This is the control room: live generation, open alerts, device health and an hourly energy curve. Nemal will show where this data comes from.

Slide 3 · 0:45–1:15 · Nemal

On screen: Assets page: switch tabs, open Add, then cancel

Nemal says: Thank you, Charith. The Asset Service owns the registry: five facilities, eleven assets and thirty-two sensors across solar, wind, hydro, battery and tidal. Lists are searched and paged on the server, and every form is validated, so a blank name is rejected. Only administrators and managers can change data. Every other service refers to these assets by ID. Govindu will show them producing live data.

Slide 4 · 1:15–2:00 · Govindu

On screen: Live monitoring page, then the toast and the Alerts page

Govindu says: Thanks, Nemal. The Monitoring Service receives a reading from every sensor and checks it against our alert rules. This tile is a live sensor and the dashed line is its threshold. The simulator has just injected a fault, and here is the alert, instantly. One alert is stored, one event goes to RabbitMQ, and the browser is updated over WebSocket. That same event is now reaching Thurunu.

Slide 5 · 2:00–2:35 · Thurunu

On screen: Maintenance board: newest card, Assign, Start, Complete

Thurunu says: Yes. The Maintenance Service received the same event and, because the severity is high, it opened a corrective work order by itself. Here it is on the board. I assign a technician, start the work and complete it. Each step is its own endpoint, and the finished job is saved to the maintenance history. Yasiru will show how people are informed.

Slide 6 · 2:35–3:05 · Yasiru

On screen: Notifications page

Yasiru says: The Notification Service read the same alert from its own RabbitMQ queue. In this history, the in-app push was delivered instantly, and every email attempt is stored with its status, retry count and reason. A scheduler retries up to three times, I can retry any message here, and failing messages go to a dead-letter queue, so nothing is lost. Sachinthana will turn all this into reports.

Slide 7 · 3:05–3:45 · Sachinthana

On screen: Analytics page: pick a facility, Generate, then export CSV

Sachinthana says: The Analytics Service calls the other services over REST and never touches their databases. For this facility it integrates average power into kilowatt-hours for each hour, and calculates availability, efficiency and capacity factor. I generate an energy report for the last day and export it as CSV for management. Executives see all of this read-only. Charith will close with security.

Slide 8 · 3:45–4:15 · Charith

On screen: Sign out, sign in as executive@reoagms.com, show the read-only pages

Charith says: Security is enforced at the gateway. Five roles carry different rights. I sign in as the executive: there are no action buttons, and the API itself answers 403 for any write. Every login and user change is recorded in the audit log. Quality is built in as well, with sixty test methods across the services.

Slide 9 · 4:15–4:50 · All members

On screen: Grafana tab, then back to the slide

Charith says: Operations are visible in Grafana, where Prometheus scrapes all seven services.

Nemal says: Validation and tests protect every API.

Govindu says: Rules are data, so thresholds change without code.

Thurunu says: Every fault ends as recorded maintenance.

Yasiru says: Every notification is stored and can be retried.

Sachinthana says: Reports turn operations into decisions.

Slide 10 · 4:50–5:00 · All members

On screen: Thank you slide

All says: Thank you. We are happy to take your questions.